

Renode Lab on GitHub Codespaces

Eight labs from a 5-min ARM MMIO warm-up to a custom C# peripheral — emulated entirely in your browser.

Welcome. Over the next few hours you will simulate eight different embedded systems — from a 5-minute MMIO taste on a bare-metal ARM Cortex-A9, through a Cortex-M4 microcontroller and a multi-core RISC-V Linux SoC, all the way to a SiFive FE310 with timer interrupts, a Robot-driven CI suite, and a peripheral you write yourself in C# — all inside a browser tab, with **nothing installed on your laptop**.

This document walks you through the setup once. After that, everything you need is in the per-lab `README.md` files in the repository.

1. What you need before starting

1.1 A personal GitHub account

If you don't already have one, create a **free personal account** at <https://github.com/join>. School-issued accounts work too, but a personal account is preferred so the Codespace and any edits you make follow you after the course ends.

GitHub's free tier includes:

Resource	Free quota / month
Codespaces compute	120 core-hours ($\approx$ 60 hours on a 2-core machine)
Codespaces storage	15 GB-month

The whole renode-lab fits comfortably in those limits if you **stop your Codespace** when you take a break and **delete it** when you're done for the day. Leaving a Codespace running idle burns compute hours; leaving it created-but-stopped burns storage. We'll cover both at the end.

1.2 A modern web browser

Chrome, Firefox, Safari, or Edge — anything from the last two years. You will spend the entire lab in two browser tabs:

1. The **VS Code** tab that GitHub auto-opens (your terminal + editor live here).
2. The **noVNC desktop** tab on port 6080 (Renode's GUI analyzers — UART windows, GPIO/LED indicators, etc.).

1.3 Working knowledge of basic Linux

You should be comfortable doing all of these from a terminal without looking them up:

- Navigating: `pwd`, `cd`, `ls`, `ls -la`
- Reading files: `cat`, `less`, `head`, `tail`, `tail -f`
- Editing files: `nano`, `vim`, or just VS Code's editor pane
- Pipes and redirection: `|`, `>`, `>>`, `2>&1`
- Background jobs: `&`, `Ctrl-C`, `Ctrl-Z`, `jobs`, `fg`
- Inspecting processes: `ps`, `pgrep -af <name>`

- Reading exit codes and recognising errors

You do **not** need to know Docker, Kubernetes, or anything about Codespaces internals.

1.4 Familiarity with basic SoC concepts

You should already understand, at a hand-wave level:

- A CPU **fetches** instructions from memory addresses, then **executes** them. The address it's about to fetch from is the **program counter (PC)**.
- A **bus** lets the CPU talk to memory regions and **memory- mapped peripherals** (UART, GPIO, timers, interrupt controller, etc.). Each peripheral lives at a known base address.
- A **UART** sends bytes one wire at a time; on most cores there's a "transmit holding register" and a "line status register" with a TX-FIFO-empty bit.
- A **GPIO port** has registers for direction (input/output), output data (ODR), and atomic set/reset (BSRR on STM32).
- An **ELF** is a packaged binary with a known **entry point**; on Cortex-M, after reset the CPU loads SP from address 0x0 and PC from address 0x4 (the vector table).
- A **bootloader** (BBL, OpenSBI, U-Boot) initialises the SoC enough to hand control to a kernel.
- A **kernel** schedules processes; a **userspace shell** like BusyBox is the program you type commands into.

Lab 01 will exercise the first four bullets, lab 02 the last three, and lab 03 lets you build the bus + RAM + UART picture from nine lines of declarative platform description.

2. First launch (only do this once)

2.1 Open the lab repository

Visit <https://github.com/prag79/renode-lab>. Sign in to your GitHub account if prompted.

2.2 Click the "Open in GitHub Codespaces" badge

Near the top of the page you'll see this badge:



Click it. GitHub will:

1. Show a "**Create codespace**" screen — accept the defaults (2-core machine, 8 GB RAM, 32 GB storage). Click **Create codespace**.
2. Spin up a small Linux VM somewhere in their cloud (≈ 30 s).
3. Pull the prebuilt image `ghcr.io/prag79/renode-lab:latest` from the GitHub Container Registry (≈ 30 – 60 s, only on first launch — afterwards it's cached).
4. Open VS Code in your browser, attached to the running container.

A second tab will auto-open at `*.app.github.dev:6080/vnc.html` — that's the noVNC desktop. If it shows "Failed to connect", give it 10 seconds and click **Connect** in the noVNC page.

2.3 Verify the environment

In the VS Code terminal (open one with **Ctrl-`** if it isn't already), type:

You should see all the available labs:

Then run lab 01:

You should land at a (STM32F4_Discovery) prompt within a couple of seconds. That's the **Renode monitor** — Renode's interactive command line. From here, type:

If the second `sysbus.cpu PC` prints a different value than the first, your environment is healthy. Type `quit` to exit.

If anything above fails, see § 5 **Troubleshooting**.

3. The eight exercises

Each lab has its own detailed `README.md` with a 7-section walkthrough — bring up, what just happened, first commands, headless UART recipes, useful monitor command tables, mini- experiments, and clean exit.

Lab	Time	What you'll do	Detailed README
00	~5 min	A 5-minute MMIO warm-up: bare-metal ARM Cortex-A9 reads/writes four 32-bit registers in a memory-mapped "SmartTimer" stub, then you read them back from the monitor. (Aliased: <code>lab demo</code> .)	<code>labs/00-Demo/README.md</code>
01	~20 min	Boot a bundled Contiki firmware on a simulated STM32F4 Discovery board.	<code>labs/01-bundled-stm32f4/README.md</code>

Lab	Time	What you'll do	Detailed README
		Read UART, single-step the CPU, blink the on-board LED from the simulator.	
02	~30 min	Boot an unmodified RISC-V Linux kernel (5 cores, OpenSBI, BusyBox userspace) on the SiFive HiFive Unleashed model. Poke around /proc, trace UART traffic at the bus level.	labs/02-linux-on-hifive/README.md
03	~45 min	Cross-compile bare-metal C for RV64. Run it on a 9-line custom SoC you can edit. Add a second peripheral with one line.	labs/03-custom-soc/README.md
04	~45 min	Bare-metal on a real SiFive FE310 (HiFive1). Drive the SiFive UART and GPIO at their datasheet addresses; blink an LED on RV32.	labs/04-sifive-fe310/README.md
05	~60 min	RISC-V machine-mode interrupts: program mtvec/mie/mstatus and the CLINT timer, then blink the LED from an interrupt handler.	labs/05-fe310-interrupts/README.md
06	~45 min	Headless CI: write a Robot Frame-	labs/06-robot-testing/README.md

Lab	Time	What you'll do	Detailed README
		work suite that boots firmware, asserts UART output, and fails the build (non-zero exit) on regressions.	
07	~75 min	Model your own peripheral: write a memory-mapped timer IP in C#, compiled by Renode at runtime, that raises an interrupt the firmware handles.	labs/07-custom-peripheral/README.md

Do them **in order** — they increase in difficulty. Lab 00 is a 5-minute sanity check that the toolchain works; 01–02 then run bundled images, 03 builds a minimal custom SoC, 04–05 move to a real SiFive chip with real peripherals and interrupts, 06 turns it all into an automated regression test, and 07 has you write a brand-new peripheral model that the CPU talks to. Each one introduces a concept the next assumes (the three Renode primitives `mach create`, `LoadPlatformDescription`, `LoadELF + start`; then real peripherals, interrupts, and automated testing).

4. Where your edits live

The `lab NN` command does not run the lab from `/labs/...` directly. On first invocation it copies the lab into a **writable scratch tree** under your home directory and runs it from there:

Path	What it is	Survives Code-space stop?	Survives container rebuild?	Survives Code-space delete?
<code>/labs/<lab-name>/</code>	Read-only image content	yes	no	no
<code>~/work/<lab-name>/</code>	Editable copy made by <code>lab NN</code>	yes	no	no
<code>/workspaces/renode-lab/</code>	The cloned git repo	yes	yes	only if <code>git push-ed</code>

Practical rules:

- If you just want to tweak a `.resc` or a `hello.c` and re-run, edit `~/work/<lab-name>/...`. The next lab `NN` reuses your edits (`cp -ru` is idempotent — it never overwrites existing files).
- If you want changes to **outlive Codespace deletion**, you must push them to a git remote you own — see § 4.1 below. **A pull request is *not* required just to keep your work**; it's only needed if you want your changes merged into upstream.

4.1 Persisting your work past Codespace deletion

`prag79/renode-lab` is read-only for you (only the maintainer can push). To save your edits, fork the repo and push to your fork. Run these commands **inside the Codespace**, in the cloned repo at `/workspaces/renode-lab/`. Step 1 is one-time; steps 2–4 are the loop you'll run any time you want to checkpoint progress.

1. One-time: fork on GitHub and rewire the remotes. The `gh repo fork --remote` flag automatically renames the existing origin (which points at `prag79/renode-lab`) to `upstream` and adds your fork as the new origin:

If `gh`: command not found — your Codespace is running an older image that doesn't have the GitHub CLI baked in. Either **rebuild the Codespace** (palette → Codespaces: Rebuild Container) to pull the latest image, which includes `gh`, or install it by hand:

The hand-install survives Codespace stop/start but is wiped on rebuild — prefer rebuilding if you can.

2. (Recommended) work on a branch, not on main. This keeps your tinkering separate from upstream so future `git pull upstream main` is conflict-free:

3. Copy your edits from `~/work/...` back into the tracked tree. `cp -ru` mirrors files only when newer:

4. Commit and push to your fork.

Visit <https://github.com/<your-username>/renode-lab/tree/my-experiments> and confirm your files are there. The Codespace itself is now disposable: `gh codespace delete -c <name>` is safe.

Get your work back on a fresh laptop / new Codespace. Open a new Codespace **from your fork** (on `<you>/renode-lab` click *Code* → *Codespaces* → *Create on my-experiments*) and your edits are already in `/workspaces/renode-lab/`.

Optional — contribute upstream. If you fixed a typo or want to share a lab improvement with everyone, open a PR:

5. Troubleshooting

Symptom	Likely cause	Fix
<code>lab 01</code> prompt appears but <code>sysbus.cpu</code> PC keeps returning <code>0x00000000</code>	You haven't typed <code>start</code> yet — the bundled <code>.resc</code> loads the ELF but doesn't release the CPU from reset	Type <code>start</code> once.
The terminal is unreadable: UART log lines stream non-stop and your typing doesn't echo	UART output is being mirrored to the console at INFO level	<code>pause</code> (type blindly, then Enter), then <code>logLevel 3</code> <code>sysbus.uart4</code> , then <code>start</code> .
noVNC tab shows “Failed to connect” or HTTP 502	Port 6080's WebSocket handshake hadn't completed when the tab opened	Wait 10 s and click Connect in the noVNC page. If still broken: in VS Code's Ports panel, right-click port 6080 → Port Visibility → Public (or sign in to the popup).
Browser tab opens but shows the noVNC welcome screen, not a desktop	URL is missing the path	Append <code>/vnc.html</code> to the URL, then Connect .
<code>pgrep -af 'Xvfb\ fluxbox\ x11vnc\ websockify'</code> shows fewer than four processes	The headless desktop didn't fully start	Run <code>/usr/local/bin/entrypoint.sh true</code> . Check the per-service logs in <code>/tmp/Xvfb_:1.log</code> , <code>/tmp/fluxbox.log</code> , <code>/tmp/x11vnc_-display_:1.log</code> , <code>/tmp/websockify.log</code> .

Symptom	Likely cause	Fix
lab does nothing or says command not found	You're in a shell where <code>/usr/local/bin</code> isn't on <code>\$PATH</code> (rare)	Run it with the full path: <code>/usr/local/bin/lab list</code> .
Codespace fails to start with "Failed to pull image"	GHCR rate-limited you or the image package is private	Visit https://github.com/prag79?tab=packages → <code>renode-lab</code> → check it's Public . Retry.
You changed something under <code>/labs/...</code> , ran <code>lab NN</code> , and your change had no effect	Edits to <code>/labs/...</code> are overlaid on a read-only image and discarded on container rebuild — the dispatcher uses <code>~/work/<lab-name>/...</code>	Edit the file under <code>~/work/<lab-name>/...</code> instead, then <code>lab NN</code> again.

6. Cost discipline (so you don't burn quota)

GitHub will not charge you anything as long as you stay inside the free tier. Two habits keep you there:

1. **Stop the Codespace when you walk away.** In the bottom-left of VS Code (or the **Codespaces** menu in your GitHub profile) click **Stop codespace**. Compute billing pauses immediately. Storage keeps ticking at ~\$0.07 / GB / month — tiny, but cumulative.
2. **Delete the Codespace when you're done for the day** — but only after committing or copying out anything you want to keep. From your laptop terminal:

```
git commit -m "my changes"
git push
lab rm
```

...or use the web UI at <https://github.com/codespaces>.

You can always recreate a Codespace from the badge in the README. The first launch is the only slow one (~60 s); after that it's instant.

7. Where to go after the labs

- The Renode user docs: <https://renode.readthedocs.io/>
- Renode's bundled platform `.repl` files for ~100 other boards: `/opt/renode/platforms/` inside your Codespace
- The Antmicro YouTube channel for SoC-modelling walkthroughs.

If you find a bug or have an improvement, open a PR against <https://github.com/prag79/renode-lab>. The full toolchain (lab content → Docker image → GHCR → Codespaces) is in this one repository and welcomes contributions.

Quick reference card (print this if you want a single-page cheat sheet):

